OFFICIAL ARCHITECTURE DOCUMENT #1

CivicTrack: Role-Based Access Control (RBAC) Architecture

Complete technical guide detailing authentication states, storage persistence, role redirection, and dashboard isolation.

1. The Role-Based Access Control (RBAC) Framework

CivicTrack employs a declarative **Role-Based Access Control (RBAC)** architecture. The authenticated user's `role` property dictates layout composition, sidebar widgets, clickable tools, and data visibility.

System Role	Authorized Persona	Default Landing View	Access Jurisdiction
`citizen`	Public Residents	`/citizen` (Citizen Dashboard)	Can only view and manage their own submitted issues. Can report new issues with GPS coordinates.
`admin`	Municipal Officials / Commissioners	`/admin` (Admin Command Center)	City-wide jurisdiction. Master triage queue, vendor assignment, AI deduplication, and hotspot clusters.
`staff`	Maintenance Contractors / Vendors	`/staff` (Vendor Field Portal)	Restricted to work orders assigned to their contracting agency. Can submit after-repair proof photos.

2. Authentication Core (`AuthContext.jsx`)

Authentication state is managed centrally via React's **Context API** (`AuthContext`) with synchronized `localStorage` persistence through `storage.js` .

Core State Variables:

- `currentUser` : Active authenticated user object (includes `user_id` , `name` , `email` , `role` , `avatar` , `department` , `phone`).
- `users` : Array of all registered accounts (pre-seeded users + newly registered accounts).
- `staffMembers` : Computed dynamically as `users.filter(u ⇒ u.role === 'staff')` .

How `login(email, password)` Executes:

1. User submits credentials on `/login` . Form event is intercepted via `e.preventDefault()` .

2. `AuthContext` queries `Storage.getUsers()` from `localStorage` to find a matching email and password.

3. If valid, sets `currentUser` in React state and synchronizes to `Storage.setCurrentUser(user)` .

4. `redirectAfterLogin(user.role)` dynamically routes user to `/admin` , `/staff` , or `/citizen` .

```
// Dynamic Role-Based Redirection in LoginRegister.jsx
const redirectAfterLogin = (userRole) ⇒ {
  if (userRole === 'admin') navigate('/admin');
  else if (userRole === 'staff') navigate('/staff');
  else navigate('/citizen');
};
```

3. Layout & Sidebar Composition Architecture

Instead of rendering a monolithic navbar with cluttered links, the interface dynamically bifurcates into tailored layout shells based on user role:

CITIZEN LAYOUT

Public & Citizen

Uses top sticky `Navbar.jsx` with links to *Home*, *Public Map*, *Track Code*, *Dashboard*, *My Reports*, and *+ Report Issue*.

ADMIN LAYOUT

Municipal Admin

Wrapped in `AdminLayout.jsx` with `AdminSidebar.jsx`. Center links in top Navbar are hidden to eliminate collision, replaced with status badge.

STAFF LAYOUT

Field Vendor

Wrapped in `StaffLayout.jsx` with `StaffSidebar.jsx`. Displays active jobs badge, field ward map, and contractor credentials.

4. The Three Dashboards Compared

Dashboard	Data Filter Logic	Primary Functions
Citizen `/citizen`	`issues.filter(i ⇒ i.user_id ≡ currentUser.user_id)`	Personal submissions archive, status KPI cards (pending, progress, resolved), duplicate alert warnings, and quick report button.
Admin `/admin`	Aggregated city-wide dataset	Master triage queue, contractor assignment modal, contractor workload monitor, Section 17 duplicate merge, Section 18 hotspot analytics.
Vendor `/staff`	`issues.filter(i ⇒ i.assigned_to ≡ currentUser.user_id)`	Assigned job list, field ward map, status transitions (`in_progress` → `resolved`), mandatory after-repair photo upload.

5. Issue Lifecycle & State Machine

Every civic problem transitions through a deterministic state machine managed in `IssueContext.jsx` :

```
[reported] (Citizen registers with GPS map & photo)
     ↓
[under_review] (Admin inspects in Master Triage)
     ↓
[assigned] (Admin dispatches to specific Contractor)
     ↓
[in_progress] (Contractor starts work on-site)
     ↓
[resolved] (Contractor uploads completion proof photo & notes)
     ↓ (Optional: if repair is substandard)
[reopened] (Citizen reopens; sent back to contractor)
```

6. Architecture Summary for Viva & Reviews

- **Single Source of Truth:** `AuthContext` manages credentials; `IssueContext` manages tickets and audit history.
- **Zero-Latency Persistence:** Browser refreshes never lose session or tickets due to automated `localStorage` synchronization.
- **User Isolation:** Citizens only view their tickets; contractors only view assigned jobs; administrators view city-wide aggregates.
- **Scroll Ergonomics:** Universal `ScrollToTop` with `history.scrollRestoration = 'manual'` ensures pages always render from top (0, 0).